React Interview Prep

Topic 12 — Custom Hooks + JSON.stringify & JSON.parse

Custom Hooks · Where to Store · Hook vs Utility · 4 Real Hooks · Composing · JSON Boundaries · All Scenarios

■ Covers	All examples from conversation — nothing skipped
■ Series	React Interview Prep — Session 12

by Akshay Chavhan • React Interview Series

1. Like You're 5 — Morning Routine Button

Every morning you do the same routine: wake up → check weather → make tea → pack bag. Instead of repeating each step, you create a 'Morning Routine' button that does all of it. That's a custom hook — bundle repeated logic into one reusable function.

2. What is a Custom Hook?

A custom hook is a JavaScript function that:

- Starts with 'use' (naming convention React enforces)
- Can call other hooks inside it (useState, useEffect, etc.)
- Returns whatever the component needs (values, functions, both)

```
// ■ Custom hook — starts with 'use', calls hooks inside
function useFetch(url) {
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
fetch(url)
.then(res => res.json())
.then(data => { setData(data); setLoading(false); })
.catch(err => { setError(err); setLoading(false); });
}, [url]);

return { data, loading, error }; // return what component needs
}

// Using in ANY component — clean!
function UserProfile({ userId }) {
const { data, loading, error } = useFetch(`/api/users/${userId}`);
if (loading) return <p>Loading...</p>;
if (error) return <p>Error!</p>;
return <h1>{data.name}</h1>;
}

function ProductPage({ productId }) {
const { data, loading } = useFetch(`/api/products/${productId}`);
// same hook, different URL ■
}
```

> ***■ Custom hooks = write logic ONCE, reuse everywhere. Each component gets its own independent state — hooks share LOGIC, not STATE.***

3. Why 'use' Prefix is Mandatory

■ Wrong

```
// ■ No 'use' prefix
function fetchData() {
const [data, setData] = useState(null);
// React won't warn if used wrongly
// Rules of Hooks NOT enforced ■
// bugs without any hints
}
```

■ Correct

```
// ■ 'use' prefix — React tracks it
function useFetchData() {
const [data, setData] = useState(null);
// ESLint enforces Rules of Hooks ■
// warns if called in loops/conditions
// safe to use anywhere
}
```

■ 'use' prefix = passport to React's world. Without it, React can't track hook calls = bugs without warnings.

4. Custom Hook vs Utility Function

■ Wrong

```
// ■ Utility function — CANNOT use hooks
function formatUser(user) {
// ■ CRASHES — Rules of Hooks violated
const [count, setCount] = useState(0);
return user.name.toUpperCase();
}

// Called as plain function:
const result = formatUser({ name: 'Akshay' });
// React: useState called outside component!
```

■ Correct

```
// ■ Custom hook — CAN use hooks
function useUserDisplay(user) {
const [count, setCount] = useState(0);
return {
displayName: user.name.toUpperCase(),
count
};
}

// Called inside component only:
const { displayName } = useUserDisplay(user);
```

Question	Answer
Does it need useState, useEffect, useRef, or any hook?	YES → Custom hook (start with 'use')
Is it pure data transformation — no hooks needed?	NO → Regular utility function in /utils/
Can utility functions use hooks?	No — crashes with 'Invalid hook call' error
Can custom hooks call other custom hooks?	Yes ■ — most powerful pattern (composing hooks)

5. Where to Store Custom Hooks — Folder Structure

```
src/
■■■ components/
■ ■■■ UserCard/
■ ■ ■■■ UserCard.jsx
■ ■ ■■■ UserCard.css
■ ■■■ SearchBar/
■ ■■■ SearchBar.jsx
■ ■■■ useSearchBar.js ← hook used ONLY by SearchBar (stays next to it)
■
■■■ hooks/ ← all SHARED custom hooks live here
■ ■■■ useFetch.js
■ ■■■ useLocalStorage.js
■ ■■■ useDebounce.js
■ ■■■ useWindowSize.js
■ ■■■ useOnClickOutside.js
■
■■■ pages/
■ ■■■ HomePage.jsx
■ ■■■ ProfilePage.jsx
■
■■■ utils/ ← pure utility functions (NO hooks inside)
■ ■■■ formatDate.js
■ ■■■ formatCurrency.js
■ ■■■ validators.js
■
■■■ context/
■■■ AuthContext.jsx
■■■ ThemeContext.jsx
```

Situation	Where to Put It
Hook used in 2+ components	src/hooks/useSomething.js (shared)
Hook used in only 1 component	src/components/MyComp/useMyHook.js (next to component)
No hooks inside — pure function	src/utils/something.js (utility)

6. useLocalStorage — Persist State to Browser

Exactly like useState but data survives page refresh.

```
function useLocalStorage(key, initialValue) {
const [value, setValue] = useState(() => {
try {
const stored = localStorage.getItem(key);
return stored ? JSON.parse(stored) : initialValue;
} catch { return initialValue; }
});
// setStoredValue is RETURNED to component — component calls it!
const setStoredValue = useCallback((newValue) => {
setValue(newValue); // update React state
localStorage.setItem(key, JSON.stringify(newValue)); // save to browser
}, [key]);
return [value, setStoredValue]; // ← component destructures this
}
function Settings() {
const [theme, setTheme] = useLocalStorage('theme', 'light');
// ↑ setStoredValue returned as setTheme
// component calls it on button click
return <button onClick={() => setTheme('dark')}>Dark Mode</button>;
}
```

7. useDebounce — Wait Until User Stops Typing

Prevents API call on every single keystroke — waits until user pauses.

```
function useDebounce(value, delay = 300) {
const [debouncedValue, setDebouncedValue] = useState(value);
useEffect(() => {
const timer = setTimeout(() => {
setDebouncedValue(value); // only update after delay
}, delay);
return () => clearTimeout(timer); // cancel on next keystroke ■
}, [value, delay]);
return debouncedValue;
}
function SearchBar() {
const [query, setQuery] = useState('');
const debouncedQuery = useDebounce(query, 500);
useEffect(() => {
if (debouncedQuery) fetch(`/api/search?q=${debouncedQuery}`);
// only fires 500ms after user STOPS typing ■
}, [debouncedQuery]);
}
```

8. useWindowSize — Track Viewport Dimensions

```
function useWindowSize() {
const [size, setSize] = useState({
width: window.innerWidth, height: window.innerHeight
});
useEffect(() => {
function handleResize() {
setSize({ width: window.innerWidth, height: window.innerHeight });
}
window.addEventListener('resize', handleResize);
return () => window.removeEventListener('resize', handleResize); // cleanup ■
}, []);
return size;
}
function Layout() {
const { width } = useWindowSize();
return width > 768 ? <DesktopNav /> : <MobileNav />;
}
```

9. useOnClickOutside — Close Dropdowns/Modals

```
function useOnClickOutside(ref, handler) {
useEffect(() => {
function handleClick(event) {
// if click was OUTSIDE the ref element → call handler
if (ref.current && !ref.current.contains(event.target)) {
handler();
}
}
document.addEventListener('mousedown', handleClick);
return () => document.removeEventListener('mousedown', handleClick);
}, [ref, handler]);
}
function Dropdown() {
const [isOpen, setIsOpen] = useState(false);
const dropdownRef = useRef(null);
useOnClickOutside(dropdownRef, () => setIsOpen(false)); // magic ■
return (
<div ref={dropdownRef}>
<button onClick={() => setIsOpen(!isOpen)}>Menu</button>
{isOpen && <ul>...</ul>}
</div>
);
}
```

10. Composing Custom Hooks — Hooks Calling Hooks

Custom hooks can call other custom hooks — one of the most powerful patterns.

```
// useUserSearch = useDebounce + useFetch combined
function useUserSearch(query) {
const debouncedQuery = useDebounce(query, 400); // custom hook
const { data, loading, error } = useFetch( // custom hook
debouncedQuery ? `/api/users?q=${debouncedQuery}` : null
);
return { users: data, loading, error };
}
```

```
// Component uses ONE clean hook instead of two
function SearchPage() {
const [query, setQuery] = useState('');
const { users, loading } = useUserSearch(query); // ■ so clean
return (
<>
<input onChange={e => setQuery(e.target.value)} />
{loading ? <Spinner /> : users?.map(u => <p key={u.id}>{u.name}</p>)}
</>
);
}
```

> ■ ***Each hook instance is independent — same hook in 3 components = 3 separate states. Hooks share LOGIC not STATE.***

11. JSON.stringify & JSON.parse — Complete Clarity

This came up in useLocalStorage — let's understand it fully once and for all.

The Core Problem

localStorage can ONLY store STRINGS. That's it. Nothing else.

```
localStorage.setItem('name', 'Akshay'); // ■ string — works
localStorage.setItem('age', 25); // ■■ becomes '25' (string)
localStorage.setItem('user', { name: 'Akshay' }); // ■ stored as '[object Object]'

localStorage.getItem('user'); // '[object Object]' ← USELESS ■
```

> ■ ***localStorage is a string-only store. Objects, arrays, numbers, booleans — all need conversion before storing.***

JSON.stringify — Object → String

```
JSON.stringify({ name: 'Akshay', age: 25 });
// → '{"name":"Akshay","age":25}' ← a STRING now ■
JSON.stringify([1, 2, 3]);
// → '[1,2,3]'
JSON.stringify(true); // → 'true'
JSON.stringify(42); // → '42'
```

JSON.parse — String → Object

```
JSON.parse('{"name":"Akshay","age":25}');
// → { name: 'Akshay', age: 25 } ← object again ■
JSON.parse('[1,2,3]'); // → [1, 2, 3]
JSON.parse('true'); // → true (boolean)
JSON.parse('42'); // → 42 (number)
```

Why Both Are Needed Together

```
const user = { name: 'Akshay', role: 'Developer' };

// SAVING — object must become string first
localStorage.setItem('user', JSON.stringify(user));
// stored: '{"name":"Akshay","role":"Developer"}'

// READING — string must become object again
const stored = localStorage.getItem('user'); // still a string!
const parsedUser = JSON.parse(stored); // now an object ■

console.log(parsedUser.name); // 'Akshay' ■
console.log(stored.name); // undefined ■ — it's a string!
```

12. ALL Places You See This Pattern

Not just localStorage. Any time data crosses a BOUNDARY that only understands strings.

1. localStorage / sessionStorage

```
localStorage.setItem('cart', JSON.stringify(cartItems)); // save
const cart = JSON.parse(localStorage.getItem('cart')); // read
```

2. API Calls — Sending Data (fetch body)

■ Wrong

```
// ■ Object → becomes '[object Object]'
fetch('/api/users', {
method: 'POST',
body: { name: 'Akshay' } // wrong!
});
```

■ Correct

```
// ■ Stringify for the wire
fetch('/api/users', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ name: 'Akshay' })
});
```

3. API Calls — Receiving Data (res.json() does it automatically)

```
const res = await fetch('/api/users');
const data = await res.json(); // internally calls JSON.parse for you ■
// res.text() → raw string | res.json() → parsed object (use this!)
```

4. Deep Clone an Object — Quick Trick

```
const original = { name: 'Akshay', skills: ['React', 'Node'] };
const deepClone = JSON.parse(JSON.stringify(original)); // full copy ■
deepClone.skills.push('MongoDB');
console.log(original.skills); // ['React', 'Node'] — unchanged ■
// ■■ This trick BREAKS with:
// Functions → get DROPPED | undefined → gets DROPPED | Date → becomes string
// Use structuredClone() for a proper deep clone instead
```

5. Cookies

```
document.cookie = `user=${JSON.stringify(user)}; path=/`; // save
const user = JSON.parse(cookieValue); // read
```

13. What Gets LOST in JSON.stringify

Value type	What happens in JSON.stringify
Function () => {}	■ DROPPED — completely removed from output
undefined	■ DROPPED — key and value both removed
Symbol	■ DROPPED — completely removed
Date object	■■ Becomes a STRING — loses Date methods
NaN	■■ Becomes null

Infinity / -Infinity	■■ Becomes null
String, Number, Boolean, null	■ Preserved correctly

> ■ ***Always wrap JSON.parse in try/catch when reading from localStorage or external sources. Corrupt data throws SyntaxError and crashes your app.***

```
// ■ Safe pattern — always use try/catch
try {
const data = JSON.parse(localStorage.getItem('user'));
} catch (error) {
console.error('Corrupt data in localStorage:', error);
localStorage.removeItem('user'); // clear bad data
}
```

14. Complete Reference Table

Scenario	Method	Direction
Save to localStorage	JSON.stringify()	object → string
Read from localStorage	JSON.parse()	string → object
Send data via fetch body	JSON.stringify()	object → string
Receive data via fetch	res.json() (auto)	string → object
Deep clone (simple)	JSON.parse(JSON.stringify())	object→string→object
Cookie storage	JSON.stringify() + JSON.parse()	both directions
URL query params	JSON.stringify() + JSON.parse()	both directions

> ■ ***Any time data crosses a BOUNDARY that only understands text: Going OUT = JSON.stringify() (pack it) Coming IN = JSON.parse() (unpack it) localStorage, API body, URL, cookies = string-only boundaries.***

15. Interview Q&A;

Q Beginner **Q1. What is a custom hook?**

➜ A custom hook is a JavaScript function whose name starts with 'use' and can call other hooks inside it. It lets you extract component logic into reusable functions — instead of duplicating useState, useEffect across components, you write the logic once in a custom hook and reuse it anywhere.

Q Beginner **Q2. Why must custom hooks start with 'use'?**

➜ React's ESLint plugin uses the 'use' prefix to identify functions as hooks and enforce the Rules of Hooks — like only calling hooks at the top level and not inside conditions. Without the prefix, the linter won't warn you when you misuse hooks, leading to hard-to-find bugs.

Q Beginner **Q3. Do custom hooks share state between components?**

➜ No — custom hooks share logic, not state. Each component that calls the same custom hook gets its own independent state. It's the same as calling useState twice — both get separate state. If you need shared state across components, use Context or a global store like Zustand.

Q Intermediate **Q4. What is the difference between a custom hook and a utility function?**

➜ A utility function is a plain JS function — it cannot use hooks. A custom hook looks like a function but can use any React hooks inside it. If your logic needs useState, useEffect, useRef — it must be a custom hook. If it's pure data transformation with no hooks — use a utility function.

Q Intermediate **Q5. Where should custom hooks be stored in a React project?**

➜ Hooks used across 2+ components go in src/hooks/. Hooks used only by one specific component live next to that component in its folder. Pure functions with no hooks go in src/utils/. This keeps the codebase organised and makes it clear what needs React vs what doesn't.

Q Intermediate **Q6. Why do we need JSON.stringify when saving to localStorage?**

➜ localStorage can only store strings. If you save an object directly, it gets stored as '[object Object]' which is useless. JSON.stringify converts any JS value to a JSON string. JSON.parse converts it back when reading. Together they are the save/load pair for any non-string data.

Q Expert **Q7. Can a custom hook call another custom hook?**

➜ Yes — and this is one of the most powerful patterns. You can compose small focused hooks into larger ones. For example, a useUserSearch hook can internally call useDebounce and useFetch, combining their logic into one clean API. This is how you build layered, reusable abstractions.

Q Expert

Q8. What gets lost when you use JSON.stringify?

➜ Functions are dropped completely, undefined values are dropped, Symbols are dropped, Date objects become strings losing their Date methods, NaN and Infinity become null. Always use try/catch around JSON.parse since corrupt data throws SyntaxError. Use structuredClone() for a proper deep clone that handles more types.